James Hyle

CS450: Programming Language Translation

Dr. Sherman

4/14/26

Milestone 4: ANTLR Project - Bytecode Generation

What didn't work:

I struggled quite a bit with this checkpoint as well. While the low levels of translation is cool, it felt a lot different than the other work we have been doing with compilers. Some of the code was easy to spot and implement, for example PostFix expressions were straightforward, but most of the trouble I ran into was FieldAccesses and TypeDeclarations. Both of these are harder I think because we have to keep track of more variables on the stack as the program runs. Another thing I struggled with was the exceptions generated from failing tests, they were pretty cryptic and took a bit of time to unfold. I tried to put debugging statements in, but they are hard to track inside the bytecode. I did have to use AI tools more than I liked on this checkpoint, but it was mostly putting in error messages to figure out what the exception stack was telling me. I also tried to put a few of the test case files into the ASM Plugin, but it wasn't super helpful for me. I don't know if I was missing a setting, but I could not figure out how the generated code was matching to our higher level of code generation.

What did work:

As I mentioned above some of the visitors were more straightforward to implement. Another couple of easy implementations were ConditionalExpressions and DoStatements, I think these are easier for me to picture in my head as they are more familiar programming constructs. I can more easily follow the flow of logic inside, and there are not as many details to check for. I struggle with the abstraction of visitors such as Class and Array Creation or Initializers as I have never had to think about those

constructs as deeply as we do in this class. I have mentioned this before in other reports, but it does make me appreciate how monumental of a task it is to fully understand and translate programming languages.

What I learned:

I definitely feel more comfortable with manipulating the runtime stack after this checkpoint and I can see more clearly how our programs are built up. I spent more time than I should have debugging tests and researching, but that is valuable information. I found one source to be pretty good for getting the hang of things at this site, https://asm.ow2.io/asm4-guide.pdf. I found quite a bit of useful details and examples inside, explicit explanations of opcodes, arguments, and keywords really helped me put the scope of the project together in my mind, although I probably underestimated it a bit. I think it is a great resource for the test coming up, and its examples helped me tremendously. I finished this checkpoint with 80/86 tests passing which I am happy with as I spent a lot of time on the project, both on finishing the TypeChecker code and the ByteCodeGenerator classes.